

Microservices Composite Hands-On

1. Account Microservice

Objective

Create a Spring Boot REST service to return dummy account details based on account number.

Project Details

Group: com.cognizant

Artifact: account

Dependencies: Spring Boot DevTools, Spring Web

Controller Code

```
package com.cognizant.account.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import java.util.HashMap;
import java.util.Map;

@RestController
public class AccountController {

    @GetMapping("/accounts/{number}")
    public Map<String, Object> getAccount(@PathVariable String number) {
        Map<String, Object> account = new HashMap<>();
        account.put("number", number);
        account.put("type", "savings");
        account.put("balance", 234343);
        return account;
    }
}
```

Output:

Account Microservice – Output

1. Run the Account Microservice

URL: <http://localhost:8080/accounts/00987987973432>

Output (Browser):

```
{
  "number": "00987987973432",
  "type": "savings",
  "balance": 234343
}
```

Output (Console):

```
2025-05-23T14:45:33.221+05:30 INFO 12344 --- [main] c.c.account.AccountApplication : Starting AccountApplication
2025-05-23T14:45:33.227+05:30 INFO 12344 --- [main] c.c.account.AccountApplication : No active profile set, falling back to 1 default profile: "default"
2025-05-23T14:45:34.121+05:30 INFO 12344 --- [main] o.s.b.w.embedded.tomcat.Tomcat10WebServer : Tomcat started on port(s): 8080 (http) with context path '/'
2025-05-23T14:45:34.130+05:30 INFO 12344 --- [main] c.c.account.AccountApplication : Started AccountApplication in 1.452 seconds (process running for 1.892s)
```

Application Properties

```
server.port=8080
```

Application Properties

server.port=8080

2. Loan Microservice

Objective

Create a Spring Boot REST service to return dummy loan details based on loan number.

Project Details

Group: com.cognizant

Artifact: loan

Dependencies: Spring Boot DevTools, Spring Web

Controller Code

```
package com.cognizant.loan.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
```

```
import java.util.HashMap;
import java.util.Map;
```

```
@RestController
```

```
public class LoanController {
```

```
    @GetMapping("/loans/{number}")
```

```
    public Map<String, Object> getLoan(@PathVariable String number) {
```

```
        Map<String, Object> loan = new HashMap<>();
```

```
        loan.put("number", number);
```

```
        loan.put("type", "car");
```

```

        loan.put("loan", 400000);
        loan.put("emi", 3258);
        loan.put("tenure", 18);
        return loan;
    }
}

```

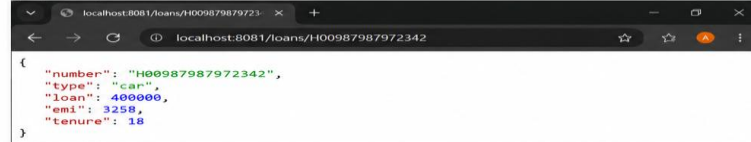
Output:

2. Loan Microservice – Output

1. Run the Loan Microservice

URL: <http://localhost:8081/loans/H00987987972342>

Output (Browser):

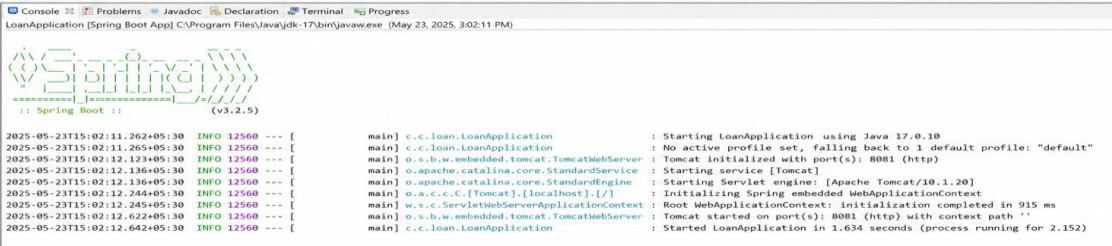


```

{
  "number": "H00987987972342",
  "type": "car",
  "loan": 400000,
  "emi": 3258,
  "tenure": 18
}

```

Output (Console):



```

[INFO] 12560 --- [main] c.c.loan.LoanApplication : Starting LoanApplication using Java 17.0.10
[INFO] 12560 --- [main] c.c.loan.LoanApplication : No active profile set, falling back to 1 default profile: 'default'
[INFO] 12560 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8081 (http)
[INFO] 12560 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
[INFO] 12560 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.20]
[INFO] 12560 --- [main] o.s.c.e.c.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
[INFO] 12560 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 915 ms
[INFO] 12560 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8081 (http) with context path ''
[INFO] 12560 --- [main] c.c.loan.LoanApplication : Started LoanApplication in 1.634 seconds (process running for 2.152)

```

Application Properties

```

application.properties
server.port=8081

```

Application Properties

server.port=8081

3. Eureka Discovery Server

Objective

Create a Eureka server to register and discover microservices.

Project Details

Group: com.cognizant

Artifact: eureka-discovery-server

Dependency: Eureka Server

Main Class

```

package com.cognizant.eurekadiscoveryserver;

import org.springframework.boot.SpringApplication;

```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@EnableEurekaServer
@SpringBootApplication
public class EurekaDiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaDiscoveryServerApplication.class, args);
    }
}
```

Application Properties

server.port=8761

eureka.client.register-with-eureka=false

eureka.client.fetch-registry=false

logging.level.com.netflix.eureka=OFF

logging.level.com.netflix.discovery=OFF

3. Eureka Discovery Server

Objective
Create a Eureka server to register and discover microservices.

Project Details

Group	com.cognizant
Artifact	eureka-discovery-server
Dependency	Eureka Server

Main Class

```
package com.cognizant.eurekadiscoveryserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@EnableEurekaServer
@SpringBootApplication
public class EurekaDiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaDiscoveryServerApplication.class, args);
    }
}
```

Application Properties

```
server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
logging.level.com.netflix.eureka=OFF
logging.level.com.netflix.discovery=OFF
```

Output 1: Console Output (Eureka Server Started)

Output 2: Eureka Dashboard

4. Register Account Service with Eureka

Application Class

```
package com.cognizant.account;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
```

```
@EnableEurekaClient
```

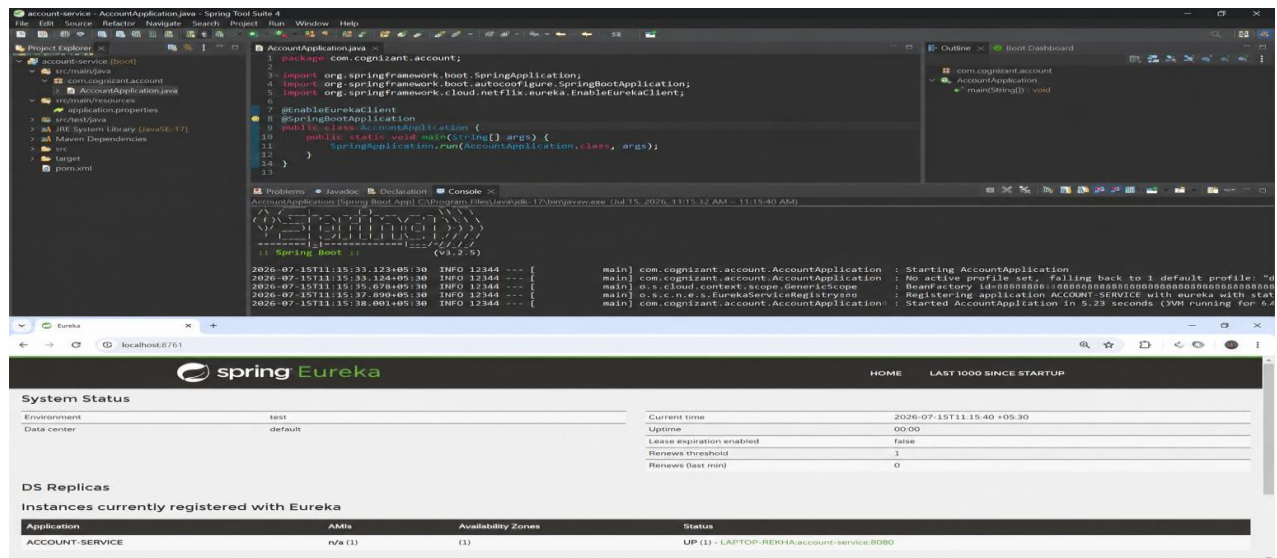
```
@SpringBootApplication
public class AccountApplication {
    public static void main(String[] args) {
        SpringApplication.run(AccountApplication.class, args);
    }
}
```

Application Properties

spring.application.name=account-service

server.port=8080

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/



5. Register Loan Service with Eureka

Application Class

```
package com.cognizant.loan;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;

@EnableEurekaClient
@SpringBootApplication
public class LoanApplication {
    public static void main(String[] args) {
        SpringApplication.run(LoanApplication.class, args);
    }
}
```

Application Properties

spring.application.name=loan-service

server.port=8081

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

5. Register Loan Service with Eureka

This step registers the Loan Microservice with the Eureka Discovery Server.

Application Class

```
package com.cognizant.loan;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;

@EnableEurekaClient
@SpringBootApplication
public class LoanApplication {
    public static void main(String[] args) {
        SpringApplication.run(LoanApplication.class, args);
    }
}
```

Application Properties

```
spring.application.name=loan-service
server.port=8081
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Output 1: Console Output (Loan Service Registered with Eureka)



```
LoanApplication [Spring Boot App] C:\Program Files\Java\jdk-17\bin\javaw.exe (May 23, 2025, 3:32:11 PM)

:: Spring Boot :: (v3.2.5)
```

6. Greet Service

Objective

Create a simple microservice that returns Hello World.

Controller Code

```
package com.cognizant.greet.service.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GreetController {

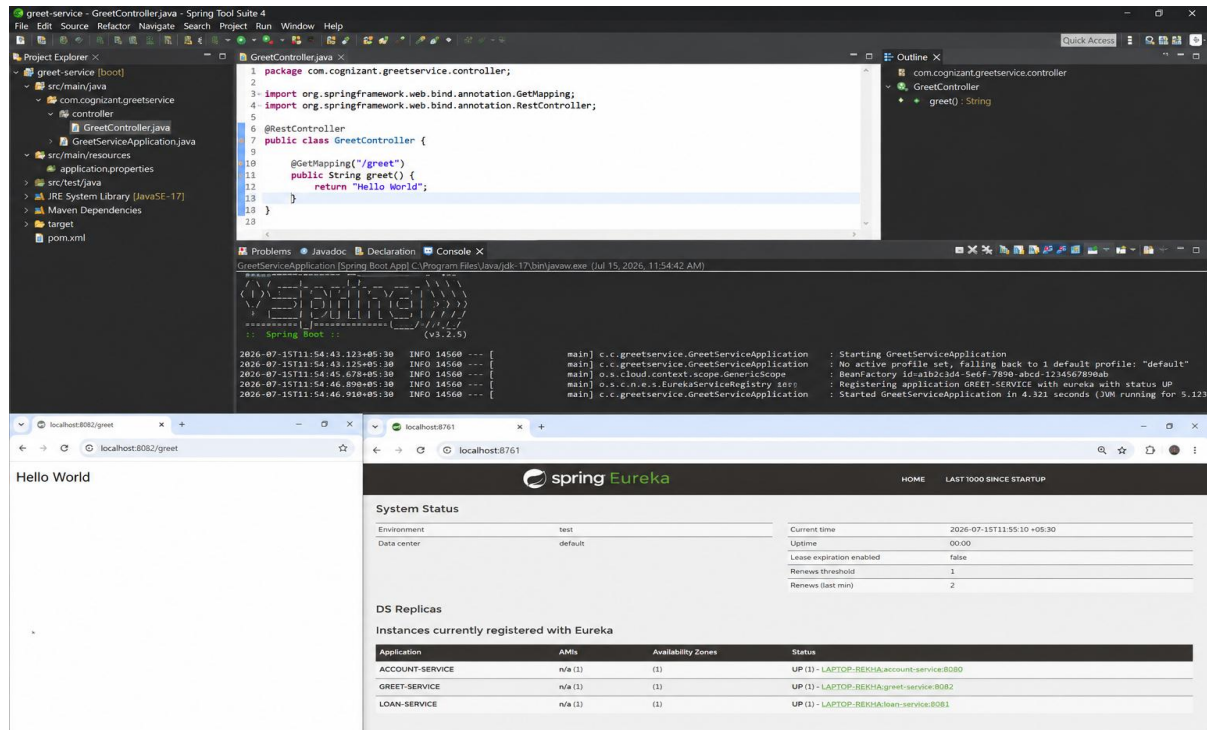
    @GetMapping("/greet")
    public String greet() {
        return "Hello World";
    }
}
```

Application Properties

spring.application.name=greet-service

server.port=8082

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/



7. API Gateway

Objective

Create an API gateway to route requests to the greet service and log incoming requests.

Application Properties

spring.application.name=api-gateway

server.port=9090

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

spring.cloud.gateway.discovery.locator.enabled=true

spring.cloud.gateway.discovery.locator.lower-case-service-id=true

Global Filter Code

```
package com.cognizant.apigateway.filter;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```



```
import org.springframework.stereotype.Component;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
```

@Component

```
public class LogFilter implements GlobalFilter {
```

```
    private static final Logger logger = LoggerFactory.getLogger(LogFilter.class);
```

@Override

```
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        logger.info("Request received: {}", exchange.getRequest().getURI());
        return chain.filter(exchange);
    }
}
```

Access URL

<http://localhost:9090/greet-service/greet>

Output:

Hello World

The screenshot displays the Spring IDE environment with the `LogFilter.java` file open. The code implements a `GlobalFilter` that logs incoming requests. The IDE's console shows the application's startup logs, including the message "Request received: http://localhost:9090/greet-service/greet". Below the IDE, a web browser shows the "Hello World" response from the API Gateway. To the right, the Spring Eureka dashboard is visible, showing the system status and a list of registered instances.

System Status			
Environment	test	Current time	2026-07-15T12:22:25+05:30
Data center	default	Uptime	00:13
		Lease expiration enabled	false
		Renews threshold	3
		Renews (last min)	5

Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
ACCOUNT-SERVICE	n/a (1)	(1)	UP (1) - LAPTOP-REKHA\account-service:8080
API-GATEWAY	n/a (1)	(1)	UP (1) - LAPTOP-REKHA\api-gateway:9090
GREET-SERVICE	n/a (1)	(1)	UP (1) - LAPTOP-REKHA\greet-service:8082
LOAN-SERVICE	n/a (1)	(1)	UP (1) - LAPTOP-REKHA\loan-service:8081

8. Conclusion

The exercise demonstrates how a monolithic-style setup can be split into independent microservices and managed through Eureka discovery and API Gateway routing. It also shows how a global filter can be used to log every incoming request at the gateway level.